

Estruturas de Decisão em C++

1. Introdução às estruturas de decisão

Um programa normalmente não executa sempre as mesmas instruções da mesma maneira. Dependendo dos dados fornecidos pelo usuário, de resultados calculados ou de determinadas situações, o programa pode precisar escolher entre diferentes caminhos de execução.

As estruturas de decisão permitem que um programa tome decisões com base em condições.

Por exemplo:

- verificar se um aluno foi aprovado ou reprovado;
- verificar se uma pessoa é maior ou menor de idade;
- determinar o desconto de uma compra;
- verificar se uma senha está correta;
- escolher uma operação em uma calculadora;
- apresentar diferentes opções de um sistema;
- classificar um produto, temperatura ou nota;
- validar se uma determinada operação pode ser realizada.

Sem estruturas de decisão, o programa teria pouca capacidade de reagir de maneira diferente para diferentes situações.

Em C++, as principais estruturas de decisão estudadas nesta apostila são:

- if → verifica uma condição simples;
- if...else → permite escolher entre dois caminhos;
- if...else if...else → permite trabalhar com várias condições;
- switch...case → permite realizar uma seleção entre vários valores.

A escolha adequada entre essas estruturas contribui para tornar o código mais organizado, legível e fácil de manter.

2. IF – Estrutura condicional ou de decisão

A estrutura if permite executar um bloco de código somente quando uma determinada condição for verdadeira.

- if → verifica uma condição simples.
- if...else → apresenta dois caminhos possíveis.
- if...else if...else → permite testar várias condições em sequência.

2.1 Sintaxe básica do if

```
if (condicao) {  
    // bloco de código executado  
    // se a condição for verdadeira  
}
```

A condição é uma expressão que pode resultar em true (verdadeiro) ou false (falso).

Por exemplo:

```
if (idade >= 18) {  
    cout << "Maior de idade";  
}
```

Nesse caso, o bloco será executado somente se idade $\geq$ 18 for verdadeiro.

3. IF...ELSE

O if...else permite trabalhar com dois caminhos possíveis.

```
if (condicao) {  
    // executa se a condição for verdadeira  
} else {  
    // executa se a condição for falsa  
}
```

Exemplo

```
#include <iostream>  
using namespace std;  
int main() {  
  
    int idade;  
  
    cout << "Digite sua idade: ";  
    cin >> idade;  
  
    if (idade >= 18) {  
        cout << "Maior de idade." << endl;  
    } else {  
        cout << "Menor de idade." << endl;  
    }  
  
    return 0;  
}
```

Se o usuário informar 20:

```
Maior de idade.
```

Se informar 15:

```
Menor de idade.
```

4. IF...ELSE IF...ELSE

Quando existem várias possibilidades, podemos utilizar else if.

```
if (condicao1) {  
    // executa se condicao1 for verdadeira  
}  
else if (condicao2) {  
    // executa se condicao2 for verdadeira  
}
```

```
else {  
    // executa se nenhuma condição anterior for verdadeira  
}
```

Exemplo: classificação por idade

```
#include <iostream>  
using namespace std;  
int main() {  
  
    int idade;  
  
    cout << "Digite sua idade: ";  
    cin >> idade;  
  
    if (idade < 12) {  
        cout << "Crianca." << endl;  
    }  
    else if (idade < 18) {  
        cout << "Adolescente." << endl;  
    }  
    else {  
        cout << "Adulto." << endl;  
    }  
  
    return 0;  
}
```

Nesse exemplo, existem três possibilidades:

idade < 12	→ Criança
idade < 18	→ Adolescente
caso contrário	→ Adulto

4.1 Uso das chaves { }

Em C++, as chaves delimitam o bloco de código pertencente à estrutura de decisão.

Tecnicamente, quando existe apenas uma instrução, as chaves podem ser omitidas:

```
if (idade >= 18)  
    cout << "Maior de idade";
```

Entretanto, é considerada uma boa prática manter as chaves, principalmente em códigos utilizados em projetos e atividades acadêmicas:

```
if (idade >= 18) {  
    cout << "Maior de idade";  
}
```

Isso facilita a leitura e reduz a possibilidade de erros quando novas instruções forem adicionadas posteriormente.

5. Operadores utilizados nas condições

As condições utilizadas no if podem utilizar operadores relacionais e lógicos.

Operador	Significado	Exemplo
==	igual a	idade == 18
!=	diferente de	idade != 18
>	maior que	idade > 18
<	menor que	idade < 18
>=	maior ou igual	idade >= 18
<=	menor ou igual	idade <= 18
&&	E lógico	idade >= 18 && idade <= 60
	OU lógico	idade < 18 idade >= 60
!	negação	!ativo

Exemplo utilizando &&

```
if (idade >= 18 && idade <= 60) {  
    cout << "Pessoa adulta em idade ativa." << endl;  
}
```

As duas condições precisam ser verdadeiras.

Exemplo utilizando ||

```
if (idade < 18 || idade >= 60) {  
    cout << "Fora da faixa de 18 a 59 anos." << endl;  
}
```

Nesse caso, basta uma das condições ser verdadeira.

6. Exemplo: Calculadora com if

A versão original utilizava as funções printf() e scanf() da linguagem C. Em C++, podemos utilizar cout e cin.

```
#include <iostream>  
using namespace std;  
int main() {  
  
    char op;  
    double a, b, resultado;  
  
    cout << "Digite um numero A: ";  
    cin >> a;  
  
    cout << "Digite um numero B: ";  
    cin >> b;
```

```

cout << "Digite a operacao (+ - * /): ";
cin >> op;

if (op == '+') {
    resultado = a + b;
}
else if (op == '-') {
    resultado = a - b;
}
else if (op == '*') {
    resultado = a * b;
}
else if (op == '/') {
    if (b != 0) {
        resultado = a / b;
    }
    else {
        cout << "Erro: divisao por zero!" << endl;
        return 1;
    }
}
else {
    cout << "Operacao invalida!" << endl;
    return 1;
}

cout << "Resultado da operacao: "
    << resultado << endl;

return 0;
}

```

Observe que o programa também utiliza uma estrutura de decisão dentro de outra estrutura de decisão para verificar a divisão por zero.

7. SWITCH – Estrutura de decisão múltipla

O switch é uma estrutura de decisão múltipla.

Ele permite escolher um bloco de código com base no valor de uma expressão.

É particularmente útil quando temos uma variável que pode assumir vários valores previamente conhecidos, como:

- 1 → Cadastrar
- 2 → Consultar
- 3 → Alterar
- 4 → Excluir
- 5 → Sair

Em C++, o switch pode ser utilizado, por exemplo, com tipos como:

- int;
- char;
- enum.

7.1 Sintaxe básica

```
switch (expressao) {  
  
    case constante1:  
        // bloco de código  
        break;  
  
    case constante2:  
        // bloco de código  
        break;  
  
    case constante3:  
        // bloco de código  
        break;  
  
    default:  
        // opção não encontrada  
}  

```

8. Detalhes importantes sobre switch

Expressão

A expressão utilizada no switch deve ser compatível com seleção por valores integrais, como int, char ou enum.

case

Cada case representa uma possibilidade.

```
case 1:
```

significa que aquele bloco será executado quando a expressão possuir o valor 1.

Os valores dos case devem ser constantes.

break

O break encerra a execução do switch.

Por exemplo:

```
case 1:  
    cout << "Opcao 1";  
    break;
```

Sem o break, pode ocorrer o chamado fall-through, fazendo a execução continuar no próximo case.

default

O default é executado quando nenhum dos case corresponde ao valor informado.

```
default:  
    cout << "Opcao invalida";
```

9. Exemplo de switch utilizando int

```
#include <iostream>
using namespace std;
int main() {

    int opcao;

    cout << "MENU" << endl;
    cout << "1 - Somar" << endl;
    cout << "2 - Subtrair" << endl;
    cout << "3 - Multiplicar" << endl;

    cout << "Escolha uma opcao: ";
    cin >> opcao;

    switch (opcao) {

        case 1:
            cout << "Voce escolheu SOMAR." << endl;
            break;

        case 2:
            cout << "Voce escolheu SUBTRAIR." << endl;
            break;

        case 3:
            cout << "Voce escolheu MULTIPLICAR." << endl;
            break;

        default:
            cout << "Opcao invalida!" << endl;
    }

    return 0;
}
```

10. Exemplo de switch utilizando char

O switch também pode trabalhar com caracteres.

```
#include <iostream>
using namespace std;
int main() {

    char letra;

    cout << "Digite uma letra (a, b ou c): ";
    cin >> letra;

    switch (letra) {

        case 'a':
            cout << "Letra A escolhida." << endl;
    }
```

```

        break;

    case 'b':
        cout << "Letra B escolhida." << endl;
        break;

    case 'c':
        cout << "Letra C escolhida." << endl;
        break;

    default:
        cout << "Letra nao reconhecida." << endl;
}

return 0;
}

```

Observe que caracteres são escritos entre aspas simples:

```
'a'   'b'   'c'
```

11. Comparação: SWITCH × IF-ELSE

As duas estruturas permitem implementar decisões, mas são mais adequadas para situações diferentes.

Quando utilizar switch

Utilize switch quando for necessário comparar uma mesma expressão com vários valores constantes. É muito comum em:

- menus;
- opções de sistemas;
- códigos numéricos;
- comandos representados por caracteres.

Exemplo:

```

switch (opcao) {

    case 1:
        cout << "Soma";
        break;

    case 2:
        cout << "Subtracao";
        break;

    case 3:
        cout << "Multiplicacao";
        break;

    default:
        cout << "Opcao invalida";
}

```

Quando utilizar if-else

O if-else é mais adequado quando a condição envolve:

- intervalos;
- operadores >, <, >=, <=;
- operadores && e ||;
- condições mais complexas;
- comparações que não sejam simplesmente igualdade com valores constantes.

Exemplo:

```
if (idade < 12) {  
    cout << "Crianca";  
}  
else if (idade < 18) {  
    cout << "Adolescente";  
}  
else {  
    cout << "Adulto";  
}
```

Nesse caso, if-else é mais apropriado porque estamos trabalhando com intervalos de idade.

12. Regra prática

Uma regra simples para escolher entre as duas estruturas é:

- Muitos valores fixos para a mesma variável → switch
- Condições complexas, intervalos ou expressões lógicas → if-else

Exemplo

Escolha uma opção:

- 1 - Cadastrar
- 2 - Consultar
- 3 - Alterar
- 4 - Excluir

→ switch é uma boa escolha.

Já:

- Se a nota for maior ou igual a 7 → aprovado
- Se estiver entre 5 e 7 → exame
- Caso contrário → reprovado

→ if-else if-else é mais adequado.

13. Resumo

As estruturas de decisão são fundamentais para permitir que um programa escolha diferentes caminhos de execução.

Em C++, podemos utilizar:

Estrutura	Finalidade
if	Uma condição

if...else	Dois caminhos
if...else if...else	Várias condições
switch...case	Vários valores fixos

A escolha da estrutura deve considerar o tipo de problema que está sendo resolvido, buscando sempre produzir um código claro, organizado e fácil de compreender.

Exercícios sugeridos:

1. Faça um programa que receba três notas de um aluno, calcule e mostre a média aritmética e a mensagem que segue a tabela abaixo. Para alunos de exame, calcule e mostre a nota que deverá ser tirada no exame para aprovação, considerando que a média no exame é 6.

Média Aritmética	Mensagem
De 0 a 3 (exclusive)	Reprovado
De 3 a 7 (exclusive)	Exame
De 7 a 10 (inclusive)	Aprovado

2. Faça um programa que receba um número inteiro e verifique se esse número é par ou ímpar.
3. A prefeitura de uma cidade abriu uma linha de crédito para os funcionários estatutários. O valor máximo da prestação não poderá ultrapassar 30% do salário bruto. Fazer um algoritmo que permita entrar com o salário bruto e o valor da prestação e informar se o empréstimo pode ou não ser concedido.
4. Um comerciante comprou um produto e quer vendê-lo com um lucro de 45% se o valor da compra for menor que R\$ 20,00; caso contrário, o lucro será de 30%. Entrar com o valor do produto e imprimir o valor da venda.
5. Conceito Escolar: Leia a nota de um aluno (0 a 100) e mostre o conceito:
A (90 a 100), B (70 a 89), C (50 a 69), D (menor que 50)
6. Faça um código que leia um número de 1 a 7 e imprima o dia correspondente da semana.
7. Escreva um código que leia o código de um determinado produto e mostre a sua classificação, considerando a seguinte tabela como referência (utilize a estrutura switch case):

Código	Classificação
1	Alimento não-perecível
2, 3 ou 4	Alimento perecível
5 ou 6	Vestuário
7	Higiene pessoal
8 até 15	Limpeza e utensílios domésticos
Qualquer outro código	Inválido

8. Reescreva o código do exercício anterior utilizando as estruturas *if* ou *if-else* ou *if-else-if*

Exercícios com foco em operadores lógicos

Parte 1 — Condições simples

1. Maior de idade

Leia a idade de uma pessoa e informe se ela é maior ou igual a 18 anos.

2. Número positivo

Leia um número inteiro e informe se ele é positivo, negativo ou igual a zero.

3. Aprovação

Leia duas notas de um aluno. Calcule a média e informe “Aprovado” se a média for maior ou igual a 60; “Reprovado” caso contrário.

Parte 2 — Operador lógico && — E

4. Aprovado com frequência

Um aluno será aprovado somente se sua média for maior ou igual a 60 E sua frequência for maior ou igual a 75%. Leia os dados e informe se foi aprovado.

Objetivo: perceber que && exige que as duas condições sejam verdadeiras.

5. Número dentro de um intervalo

Leia um número inteiro e verifique se ele está entre 10 e 50, incluindo os limites.

Exemplo: `if (numero >= 10 && numero <= 50)`

6. Login

Leia um usuário e uma senha. O acesso será permitido somente se usuário = “admin” E senha = “1234”. Caso contrário, informe “Acesso negado”.

7. Entrada em um evento

A pessoa precisa ter idade maior ou igual a 18 anos E possuir ingresso. Leia a idade e possuiIngresso (true/false) e determine se a entrada será permitida.

Parte 3 — Operador lógico || — OU

8. Dia útil

Leia um número de 1 a 7 para o dia da semana. Considere 1 = domingo e 7 = sábado. Informe se é fim de semana. Utilize ||.

9. Desconto

Uma loja oferece desconto para clientes que possuem cartão da loja OU são membros do programa de fidelidade. Leia as informações e determine se terá direito ao desconto.

10. Acesso prioritário

Um cliente terá atendimento prioritário se tiver idade igual ou superior a 60 anos OU possuir alguma condição que dê direito à prioridade. Determine se terá atendimento prioritário.

Parte 4 — Operador ! — NÃO

11. Sistema indisponível

Considere `bool manutencao;`. Se o sistema não estiver em manutenção, informe “Sistema disponível”. Utilize !.

12. Usuário bloqueado

Considere `bool bloqueado;`. O usuário poderá acessar o sistema somente se não estiver bloqueado. Utilize !.

Parte 5 — Combinando && e ||

13. Bolsa de estudos

Um aluno poderá receber bolsa se tiver média ≥ 80 E renda familiar $\leq$ R\$ 2.000; OU tiver média ≥ 90 .
Represente a regra utilizando operadores lógicos.

14. Desconto especial

Uma loja oferece desconto quando o cliente é membro E possui cartão; OU quando a compra é superior a R\$ 500.
Leia membro, possuiCartao e valorCompra.

15. Permissão para dirigir

Leia a idade e informe se a pessoa pode iniciar o processo para obtenção da CNH. Considere idade ≥ 18 E documento válido. Utilize &&.

Parte 6 — Problemas mais elaborados

16. Classificação de aluno

Leia nome, média e frequência. Classifique: média ≥ 70 e frequência $\geq 75\% \rightarrow$ Aprovado; média ≥ 50 e < 70 e frequência $\geq 75\% \rightarrow$ Recuperação; qualquer outra situação $\rightarrow$ Reprovado.

17. Validação de senha

Leia uma senha numérica. Ela será válida se possuir exatamente 6 dígitos, começar com número maior ou igual a 1 e terminar com número diferente de zero.

18. Sistema bancário

Leia saldo e valorSaque. O saque será realizado somente se o valor for maior que zero E menor ou igual ao saldo.
Caso contrário, informe o motivo.

19. Eleição

Leia a idade. Menor de 16 $\rightarrow$ não pode votar; 16 ou 17 $\rightarrow$ facultativo; 18 a 69 $\rightarrow$ obrigatório; 70 ou mais $\rightarrow$ facultativo.

20. Sistema de acesso

Leia idade, usuarioValido, senhaValida e bloqueado. O acesso será permitido somente se idade ≥ 18 , usuário válido, senha válida e usuário não bloqueado. Utilize &&, ! e if/else.

Desafio 1 — Entender a lógica

Considere:

```
int idade = 20;
```

```
bool possuiIngresso = true;
```

```
bool bloqueado = false;
```

Análise as expressões antes de executar o programa:

```
idade  $\geq 18$  && possuiIngresso
```

```
idade  $< 18$  || possuiIngresso
```

```
!bloqueado
```

```
idade  $\geq 18$  && possuiIngresso && !bloqueado
```

Para cada expressão, responda:

1. Qual é o resultado?
2. O resultado é true ou false?
3. Por quê?

Desafio 2 — Monte a expressão

- a) Número entre 20 e 100.
- b) Número menor que 0 ou maior que 100.
- c) Pessoa maior de idade e com carteira de motorista.
- d) Pessoa menor de idade ou sem carteira de motorista.
- e) Usuário válido, senha válida e não bloqueado.

f) Cliente VIP ou cliente com compra acima de R\$ 1.000.

Quadro de referência — Operadores lógicos

Operador	Significado	Exemplo
&&	E — todas as condições devem ser verdadeiras	idade >= 18 && possuiIngresso
	OU — pelo menos uma condição deve ser verdadeira	vip compra >= 1000
!	NÃO — inverte o valor lógico	!bloqueado